AI代码生成率94%：我们用一个 Skill 跑通需求开发全流程

原创 腾讯程序员 腾讯技术工程 Jul 20, 2026, 2:36 AM 广东 1585 listened



作者：企业微信团队 - gomezlai

一、问题：AI 写业务代码为什么总是"差一口气"？

把"AI 辅助编码"放到企业级真实项目里，我们很快撞上一堵墙。下面这几个场景，每个移动端同学应该都不陌生：

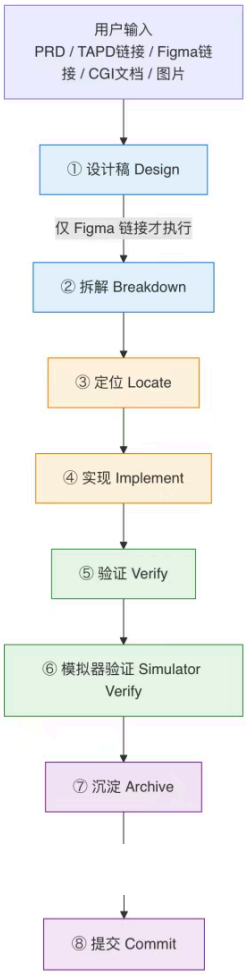
真实痛点	现象
上下文塞不下	9000+ 源文件、跨 5~6 层调用，单轮对话喂不进去
物料分散	PRD 在 TAPD、设计稿在 Figma、协议在企微文档、UI 改动还要看 Figma Token
命名不一致	用户说"邮件点击入口"，代码里其实叫 `didSelectRowAtIndexPath`
模糊指令	用户一句"按 PRD 改一下"，AI 直接跳过拆解开始改代码，越界、漏改、改错位
验证不闭环	AI 报"完成"，结果编译都没过；改完一处没顾上同步另一处
跨会话失忆	上一次的设计决策、改了哪些文件、为什么这么改，下一次会话全忘

一句话总结：**AI 不是不会写代码，是不会"按工程规范"开发需求。**

我们的解法不是换更大的模型，而是把"需求开发"这件事**流程化、原子化、可校验化**，然后把每一步都喂给 AI。

二、整体架构：把"需求开发"拆成 8 个语义化阶段

Skill 的核心是一条**严格顺序**的流水线。每个阶段输入清晰、产出明确、退出标准可机器校验。结合人日常的开发的流程，大概可以分成以下的流程：



> **子步骤命名约定**：Skill 内部统一采用「阶段·动作」式命名，例如 `设计稿·脚本筛选`、`实现·UI·切图`、`拆解·TAPD收料` ——这让 AI 在自报家门时永远清楚自己在哪一格上。

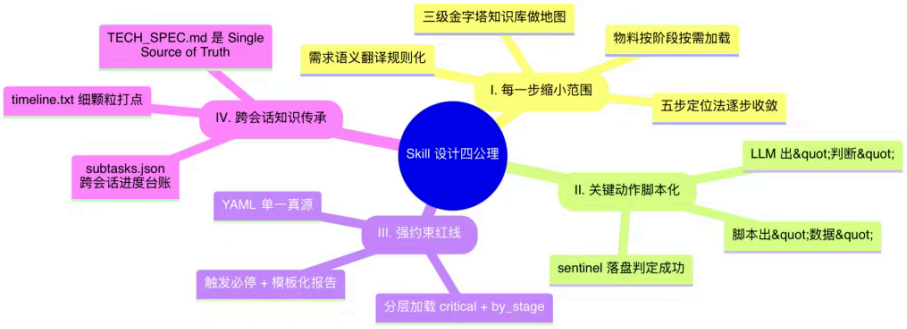
阶段	输入	关键产出	灵魂动作
① 设计稿	Figma 链接	移动端候选稿清单 + PNG 概览	**脚本化直方图筛选**，绝不允许 LLM "手感"分桶
② 拆解	PRD + 设计稿 + CGI + TAPD	五列需求清单 + `subtasks.json` 接力台账	**多源收料 + 归宿校验**（每张设计稿必须归到三类之一）
③ 定位	需求点	文件 + 行号 + 调用链	**五步定位法**（见下文）
④ 实现	调用链 + 上下文	代码改动	**自底向上**：数据 → 解析 → 枚举 → 业务 → UI → 日志
⑤ 验证	源码改动	编译报告（退出码 0）	`bazel build` **+ 最多 3 轮自修复**
⑥ 模拟器验证	编译通过的产物	装机后的截图 + 日志	"人机秒级确认 + 阶段内重试 ≤ 2 轮"
⑦ 沉淀	git diff + 时间线	`TECH_SPEC.md` 单一事实源	**跨会话知识传承的载体**

阶段	输入	关键产出	灵魂动作
⑧ 提交	全部产物	git commit + 分支	三段式 commit + AI 署名 + 代码生成率

三、第一性原理：Skill 为什么这样设计？

整条流水线背后只回答一个问题：**怎样让一个**没参与过原始实现的**AI，在新会话里像"参与过的老同事"一样把活干完？**

围绕这个目标，Skill 的设计原则可以收敛成四条公理：



下面把这四个公理逐个拆开看。

四、公理 I：每一步都在"缩小范围"——五步定位法

大模型不是搜索引擎，把整个项目 `find .` 丢给它毫无意义。Skill 把"在 9000+ 文件里找到改动点"这件事拆成 5 个收敛步骤，每步 Token 消耗严格控制。



步骤	给 LLM 的输入量	输出
1. 意图消歧	项目概述 ~2K + 用户原话	「这个目标可能对应 4 种技术解读」
2. 模块定位	目录树 + 解读结果	2-3 个候选文件路径
3. 关键词搜索	（不进 LLM）`rg` 直接跑	函数声明 + 位置
4. 调用链追踪	单个文件相关片段 ~10K	完整调用链
5. 验证确认	函数实现 ~5K	最终改动点 + 理由

> 真正的窍门：**前 2 步只看目录和文件名，第 3 步才让脚本 grep，到第 4 步才真正读代码。**一路漏斗下来，模型从来不会被整个代码库淹死。

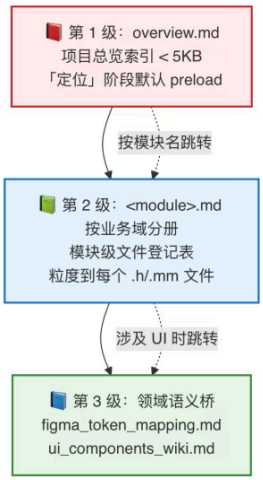
但这里还有一个**前置问题**没解决——五步定位法的第 1、第 2 步都依赖一个东西：**项目本身得有一张"AI 看得懂"的地图。**否则"项目概述 ~2K"从哪儿来？"目录树 + 解读"凭什么这么准？

下一节我们就讲：这张地图是怎么造出来、怎么维护、并且如何永不过时的。

五、代码知识库：让 AI 拥有项目的"地图"

定位精准的前提，是 AI 手里要有一份**结构化、最新、可索引**的项目知识。Skill 在这一层下了重注——我们构建了一套**三级金字塔知识库**，并配套了一个**漂移自动检测**机制，确保地图永远跟得上代码。

5.1 三级金字塔：从总览到字段，按需展开



级别	文件	粒度	加载时机
L1 总览	`project_wiki/overview.md`	模块名 + 一句话职责	「定位」阶段默认 preload（< 5KB）
L2 模块	`project_wiki/<module>.md`	每个 `.h/.mm` 文件 + 功能说明	命中模块后按需加载
L3 语义桥	`figma_token_mapping.md` / `ui_components_wiki.md`	Figma Token → 工程 API 的精确映射	「实现·UI」阶段强制参考

L1：项目总览——AI 入场的"大堂导览"

overview.md 只做一件事：用一张表告诉 AI "这个项目有哪些模块、各自负责什么"。例如：

模块	职责	详细文档
`MList/`	邮件列表展示、同步、过滤、多选编辑	`mlist.md`
`RMail/`	邮件正文渲染、附件预览、AI 总结/翻译	`rmail.md`
`CMail/`	邮件撰写、富文本编辑、附件上传、AI 润色	`cmail.md`
`Model/`	领域模型 + DB 持久化 + 业务管理器	`model.md`
...（共 N 个一级模块）		

规模示例：Model模块统计（686 个 `.h` 、456 个 `.mm` 、Top 5 大文件）。**整份文件控制在 5KB 以内**，可以毫无负担地塞进每次定位上下文。

L2：模块级——文件粒度的"街道地图"

每份 `<module>.md` 顶部有一段**机器可读的元数据**：

```
<!-- module_id: mlist -->
<!-- root_dirs:
  - App/Mailbox/MList/
-->
<!-- desc: 邮件列表展示、同步、过滤、多选编辑 -->
```

接下来是按 `Controller/ ViewModel/ View/ Helper/ Lab/` 分组的**文件登记表**，每个文件一行职责：

文件	功能说明
XYZMListController.h/.mm	**邮件列表主控制器**，管理列表展示、同步、过滤、长按、多选编辑
XYZMListViewModel.h/.mm	**邮件列表 ViewModel**，管理数据加载、分页、过滤、排序、未读数
XYZTipsView.h/.mm	邮件列表顶部提示条（同步状态、代收失败、运营活动等）
...	

> 这相当于把"老司机脑子里的项目地图"显式打印出来：哪个文件是干嘛的、它和兄弟文件什么关系——一次读 70 行就能在脑子里建立整个模块的拓扑。

L3：领域语义桥——抹平"设计 / 协议"和"代码"的鸿沟

这一层是**最容易被低估**、却**最能体现工程价值**的部分。

举个例子：设计稿上写着 `Mobile/callout` ，AI 该怎么写代码？目测字号？硬编码 `[UIFont systemFontOfSize:15]` ？——都不对。Skill 把这种翻译规则全部沉淀到 `figma_token_mapping.md` ：

```
// ❌ 错误：目测字号 + 硬编码颜色
self.titleLabel.font = [UIFont systemFontOfSize:15];
self.titleLabel.textColor = [UIColor colorWithRed:0.1 green:0.1 blue:0.1 alpha:1.0];

// ✅ 正确：按映射规则翻译 Figma Token
self.titleLabel = [UILabel xyz_styledLabel:@"callout"];       // Mobile/callout
self.titleLabel.textColor = XYZColor(base_gray_100);          // Base/base_gray_100
self.titleLabel.text = R_NSSTRING(XYZ::XXX::TITLE_KEY);       // i18n
```

整张映射表覆盖了：

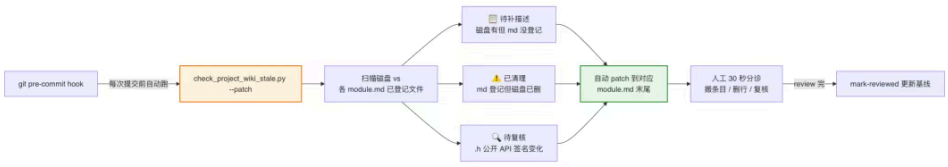
- **文字样式** `Mobile/title_1 ~ caption_2` ↔ `xyz_styledLabel:`
- **颜色** `Base/base_gray_100` ↔ `XYZColor(base_gray_100)`（自动响应 Dark Mode）
- **按钮组件** `button_blue_large` ↔ `[UIButton xyz_styledButton:...]`
- **阴影 / 渐变 / 字体兜底** 等约 20+ 类规范

⛔ **RL-29：UI 改动必须比对 `figma_token_mapping.md`，禁止硬编码字号/颜色**——这是从无数"设计稿走样"事故中淬出来的红线。

5.2 自维护：让知识库永不过时

构建知识库不难，难的是**让它不随代码漂移**。该项目半年内净增 200+ 文件、改动 1000+ 处，靠人工维护早就崩了。

Skill 的解法是一个核心脚本：** `check_project_wiki_stale.py` **。



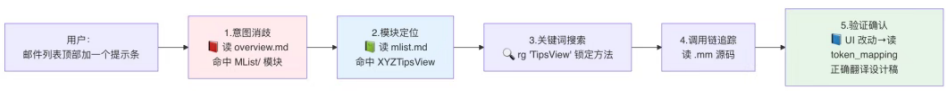
关键设计：

机制	作用
SHA 基线缓存（`.review_cache.json`）	记录每个文件上次审阅时的 SHA。再次变化时自动 flag "待复核"
三色分诊清单	新增 / 删除 / 大改三类信号分开列，30 秒就能扫完
pre-commit hook 阻断	退出码 1 = 有 stale 信号 → 阻止提交，强制开发者顺手维护
元数据驱动 overview	`<module>.md` 顶部改 `desc`，`overview.md` 索引自动跟随

> **效果**：本项目的全部模块 wiki 在过去 6 个月里没有出现过"地图和代码脱节"的情况——因为每次有人改了代码、想 commit 上去，hook 都会提醒他顺手把 wiki 同步了。

5.3 知识库 + 定位法：1 + 1 > 2

回到第四章的五步定位法，把它和知识库结合，就能看清整个**精准定位的完整闭环**：



- 第 1 步：从 L1 总览里 1 秒选出"MList"模块（不用 grep）

- 第 2 步：从 L2 模块 wiki 里 5 秒锁定 `XYZTipsView.h/.mm`（不用读源码）
- 第 3 步：进入文件后 `rg` 精准搜索（脚本而非 LLM）
- 第 4 步：只读相关片段（~10K token）
- 第 5 步：写代码前先查 L3 映射表（杜绝硬编码）

总 token 消耗从"全项目灌入"的 ~10M+ 降到 ~30K——300× 的压缩比。这就是知识库带来的本质提效。

> ✨ 一个有意思的副作用：这套知识库**对人类新人同样有用**。我们组新同学入职后，不再需要"找老人聊一上午"才知道项目结构——直接读 `overview.md` 加几份模块 wiki，半天就能上手改 bug。**"AI 友好" 和 "新人友好" 在这里完全统一了。**

但这只解决了**问题的一半**。

知识库让 AI 拥有了"代码侧的地图"——可它还要看懂"需求侧的描述"。产品同学说的"加个红点"和工程师写的 `setMailboxBadgeValue:`，中间隔着一道**语义鸿沟**：自然语言模糊、口语化、以业务视角描述；代码精确、形式化、以技术视角组织。

要让 AI 独立跑完，必须把这道鸿沟也补平。这就是下一节要讲的。

六、需求语义翻译：把"产品语言"变成"代码指令"

> 直觉上 AI 在提效，过程却强依赖于人——很大一部分"人工成本"花在了**这道翻译上**：开发者读完 PRD/Figma/CGI 后在脑子里完成"产品语言 → 代码语言"的转换，再把翻译结果喂给 AI。这一步如果不做，AI 经常会越界、漏改、改错位。Skill 在「拆解」阶段把这道翻译**规则化、可执行化**，做到 AI 也能独立完成。

6.1 鸿沟在哪？

下图是一条典型的"产品 → 代码"翻译链。每一层都可能翻车：



每一步翻车都很真实：

翻车点	真实场景	代价
① 范围错判	PRD 段落整体在讲"后台配置"，中间一句"手机上看到的效果"被当口语忽略	漏实现移动端 UI
② 归宿不明	设计稿 9 张移动端候选稿，AI 只挑 2 张做需求点，其余笼统当"参考图"	漏 7 个独立页面
③ 联想扩大	用户说"点 A 拦截"，AI 联想"按一致性 B 也应该拦"，自作主张越界	改了不该改的逻辑
④ 关键词找不到	直接 `grep "小红条"` → 0 命中；只 `grep "tips"` → 800+ 处淹没	定位失败或误命中
⑤ 找错文件	"邮件红点"翻译错位置，改了 RMail 而不是 MList	功能完全走错地方

Skill 用五个**确定性规则**逐层堵住每个翻车点。

6.2 ① 范围识别：用"硬关键词表"代替 LLM 直觉

PRD 是产品视角写的，常常 Web 后台和移动端混在一段里。让 LLM "凭语义判断"是个灾难——同一段描述里出现"配置后台"+"客户端展示"，LLM 经常因为段落主语是后台就把整段判为非移动端。

Skill 的解法是一张**强信号关键词表**，硬触发，不依赖 LLM 语义理解：

类别	关键词（命中即强制打"移动端"标签）
平台 / 端	手机上 、手机端、移动端、iOS、Android、安卓、苹果、客户端、App
原生控件 / 交互	Toast 、弹窗、浮层、小红条、红点、Tab 角标、角标、下拉刷新、侧滑、长按
iOS 系统组件	状态栏 、导航栏、Home Indicator、底部安全区、刘海
移动端页面术语	输入法 、键盘展开、全屏弹窗、actionsheet

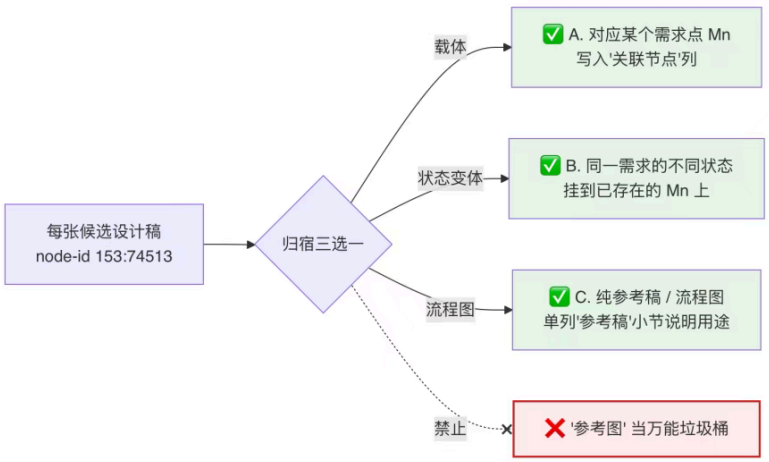
硬规则：

> 即使段落主旨在讲后端 / 配置 / 推送规则，**只要任一关键词命中**，那一段所描述的功能点就**必须单独拆成移动端项**。 范围判断不是"AI 觉得"，是"关键词命中"——客观、可机器复现、不允许降级。

这条规则非常朴素，但威力巨大：把"AI 范围错判"这种最典型的翻车，从概率事件压成 0。

6.3 ② 设计稿归宿：每张图必须归到三类之一

> ⛔ **RL-12**：候选清单里每张设计稿都必须归宿明确，不允许出现"未归类"。



关键铁律：如果某张图归不到任何需求点——

- **要么是筛选误纳**（回去把它从候选清单里去掉）

- **要么是需求点遗漏**（新增一项）

不允许用"参考图"当万能垃圾桶。这条规则把"漏需求"这种最隐蔽的事故彻底显式化。

6.4 ③ 拦截点清单：禁止"语义联想"

> ⛔ **RL-21**：任何"点击 X → 触发 Y" 类拦截，X 必须有**具体引用依据**，禁止凭语义联想扩大范围。

需求里最容易出错的是"交互拦截"。产品文档常常一句话带过，AI 最容易"自由发挥"。

Skill 强制要求输出一张**可验证的清单**：

#	触发元素 X	触发事件	响应 Y	依据来源
1	邮件列表"全选"按钮	点击	Toast 提示"超过 100 封不可全选"	`figma_overview_p3.png` 上从全选按钮指向 toast 的绿色箭头
2	顶部小红条	点击	跳转管理页	TAPD 原文："小红条点击跳转 `https://...`"

「依据来源」只接受三种：

1. **设计稿标注**：PNG 上的连接线 / 箭头从 X 指向 Y（必须有 nodeId）
2. **文档原文**：TAPD / 企微文档的**直接引用原句**
3. **用户消息**：用户原话引用

禁止用业务语义作依据：

> ❌ "Z 看起来也属于这类功能" → 删除 ❌ "为了一致性应该也拦一下" → 删除 ❌ "属于同类功能行为" → 删除

填不出具体引用的行**直接删掉，不实施**。这是一条非常硬的红线，把"AI 自作主张越界"这个公认顽疾彻底锁住。

6.5 ④ 领域联想：把"产品语言"扩展成"代码搜索词"

到这一步，我们已经把需求拆出了"M1 邮件列表顶部小红条"这样精确的需求项。但**它在代码里叫什么？**

产品同学说"小红条"，工程师在代码里可能找到的是：

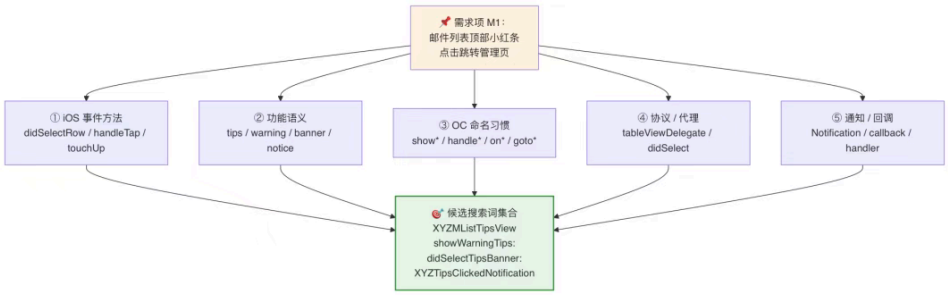
```
XYZMListTipsView          // "Tips" 才是这个组件的工程命名
XYZMListTipsType_xxx      // 枚举值
showWarningTips:                // 显示方法
```

```
is_show_warning_icon_in_mailtab // CGI 字段
XYZLOG_WARN(@"show tips")       // 日志关键字
```

"小红条"和 `Tips` / `Warning` / `Icon` 之间，隔着一道**领域知识鸿沟**——它不是 AI 不够聪明，而是产品语言和代码命名本来就属于两套词汇体系。

如果直接 `grep "小红条"`，结果一定是 0。如果只 `grep "tips"`，又会被几百处历史用法淹没。Skill 的解法是：**用 5 个搜索维度做交叉扩展**，把一个需求项展开成**一组高命中率的候选搜索词**。

5 维搜索矩阵



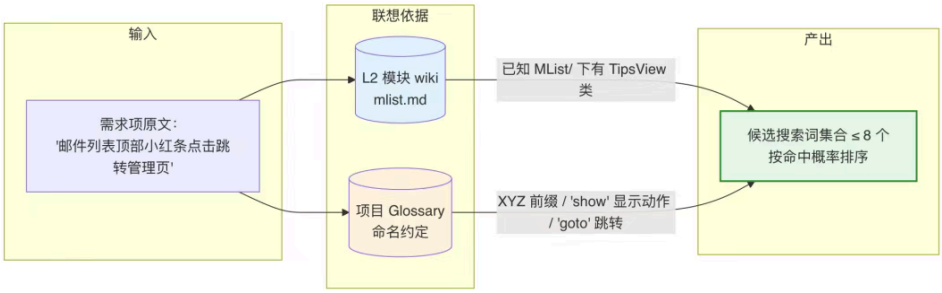
5 个维度的设计哲学：

维度	出发点	命中什么
① **iOS 事件方法**	平台标准 API	`didSelectRowAtIndexPath:` / `handleTapGesture:` / `touchUpInside:`
② **功能语义**	产品意图的英文同义词	"小红条" → `tips` / `banner` / `warning` / `notice` / `alert`
③ **OC 命名习惯**	项目里的命名前缀	`show*` / `handle*` / `on*` / `goto*` / `setup*`
④ **协议 / 代理**	谁通知谁	`tableViewDelegate` / `<XxxDelegate>` / `didSelectXxx:`
⑤ **通知 / 回调**	跨模块通信	`XxxNotification` / `XxxCallback` / `XxxHandler` / `RACSignal`

> 💡 **关键洞察**：这五个维度**不是按"和需求最相关"排，是按"代码里实际可能出现的位置"排**。① 是平台层、② 是业务层、③ 是项目命名风格层、④⑤ 是跨模块通信层——**任何一个 UI 行为，必然落在这 5 层之一**。把它当成一张"代码命名空间的全景图"，而不是凭运气联想关键词。

联想的依据：知识库 + Glossary

5 维矩阵不是凭空联想，背后有两份**领域知识**作为依据：



- **L2 模块 wiki**（第五章）告诉 AI："邮件列表模块下已经有 `XYZTipsView.h/.mm`，描述是'邮件列表顶部提示条'"——这一条直接把"小红条"翻译成了 `Tips`
- **项目 Glossary**（命名约定的总结）告诉 AI："本项目用 `show*` 表示显示、`goto*` 表示跳转、`XYZ` 是邮件插件类前缀"——这能从动词层面匹配代码命名

> 没有这两份知识，AI 联想出来的关键词是"瞎猜"；有了这两份知识，联想出来的关键词命中率 > 80%。

实战：从一句产品话到一组 grep 命令

用一个真实例子完整走一遍：

```
📝 产品原文：
   "邮件列表顶部出现红色小条，提示用户域名即将过期，
    点击跳转域名管理页"

⬇️ 第①层联想（功能语义）：
   红色小条 → tips / banner / warning / alert
   即将过期 → expire / expiry / due / warning
   跳转管理 → goto / route / push / open

⬇️ 第②层联想（项目命名风格）：
   邮件列表前缀 → XYZMList*
   提示组件类   → *TipsView / *Banner / *Notice
   跳转方法     → goto* / open* / push*

⬇️ 第③层（结合 mlist.md L2 wiki）：
   命中文件：XYZTipsView.h/.mm
   "邮件列表顶部提示条"——直接对应

⬇️ 候选搜索词集合（按命中概率从高到低）：
   1. XYZTipsView                  （强命中：组件类）
   2. showWarningTips:             （强命中：显示方法）
   3. XYZMListTipsType_            （中：枚举类型前缀）
   4. didTapTipsView:              （中：点击响应）
   5. domainExpire / domainWarning （中：业务关键词）
   6. gotoDomainManagement         （弱：跳转方法名猜测）

⬇️ 最终 grep 命令（漏斗式收敛）：
```

```
$ rg "XYZTipsView|showWarningTips" App/Mailbox/MList/ -l
App/Mailbox/MList/View/XYZTipsView.mm      ← 命中!
App/Mailbox/MList/Controller/XYZMListController.mm ← 调用方
```

> ✨ **整个过程不需要"读源码猜方法名"——只用 wiki + 命名约定就把关键词扩展出来了。从产品原文到 grep 命令，全程机器可执行。**

反例：不联想会怎么翻车？

反例	后果
直接 `grep "小红条"`	0 命中（中文 → 英文鸿沟）
只 `grep "tips"`	命中 800+ 处（项目里历史用法太多，AI 看不过来）
只 `grep "warning"`	命中错位置（项目里 "WeComKit" 也有 warning）
`grep "domain expire"`	0 命中（产品想到的业务词不在代码里出现）

5 维交叉才是唯一稳定路径——单维都会要么 0 命中、要么海量误命中。

与红线 RL-21 的边界

> ⚠️ **6.5 联想关键词**和**6.4 拦截点禁止语义联想**是两件事，不要混淆：
>
> - **6.5 允许联想**：在"找代码该改哪里"这件事上，必须用领域知识扩展候选搜索词，否则根本搜不到（这一步**只是缩小搜索范围**，不直接影响实现）
> - **6.4 禁止联想**：在"X 触发 Y 是哪条交互"这件事上，必须有具体引用依据，不能因为"看起来像"就加进拦截清单（这一步**直接决定实现内容**，关系到"AI 越界"红线）
>
> 一句话：**联想用于搜索，引用用于决策。**

6.6 ⑤ 翻译产物：五列表格 + `subtasks.json`

经过①②③ 三道关之后，需求侧的语义已经被收敛成**结构化清单**。它就是「拆解」阶段的产出：

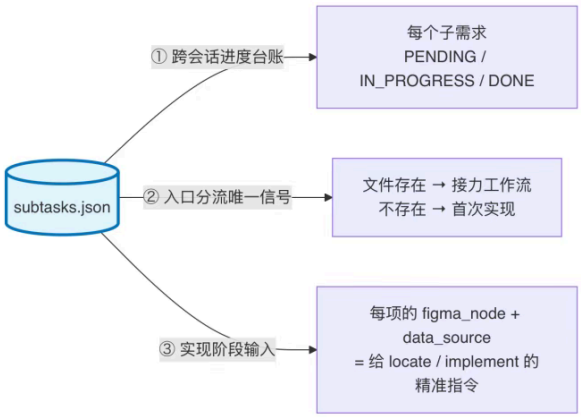
人类可读的五列表格：

序号	需求项	类型	数据来源	关联设计稿 nodeId
M1	邮件列表顶部小红条	新增 UI	CGI 字段 `is_show_warning_icon_in_mailtab`	153:74513
M2	Tab 角标显示感叹号	修改逻辑	已存字段 + 优先级判定	153:74600
M3	点击小红条跳转管理页	新增交互	TAPD 原文（已引用）	153:74521

机器可读的 `subtasks.json`（结构化字段）：

```
[
  {"id":"M1","title":"邮件列表顶部小红条","type":"新增UI",
   "data_source":"CGI字段is_show_warning_icon_in_mailtab",
   "figma_node":"153:74513","depends_on":[]},
  {"id":"M2","title":"Tab 角标显示感叹号","type":"修改逻辑",
   "data_source":"已存字段","figma_node":"153:74600","depends_on":["M1"]}
]
```

这份 JSON 是 Skill 的**关键中枢——它同时承担三个角色：**

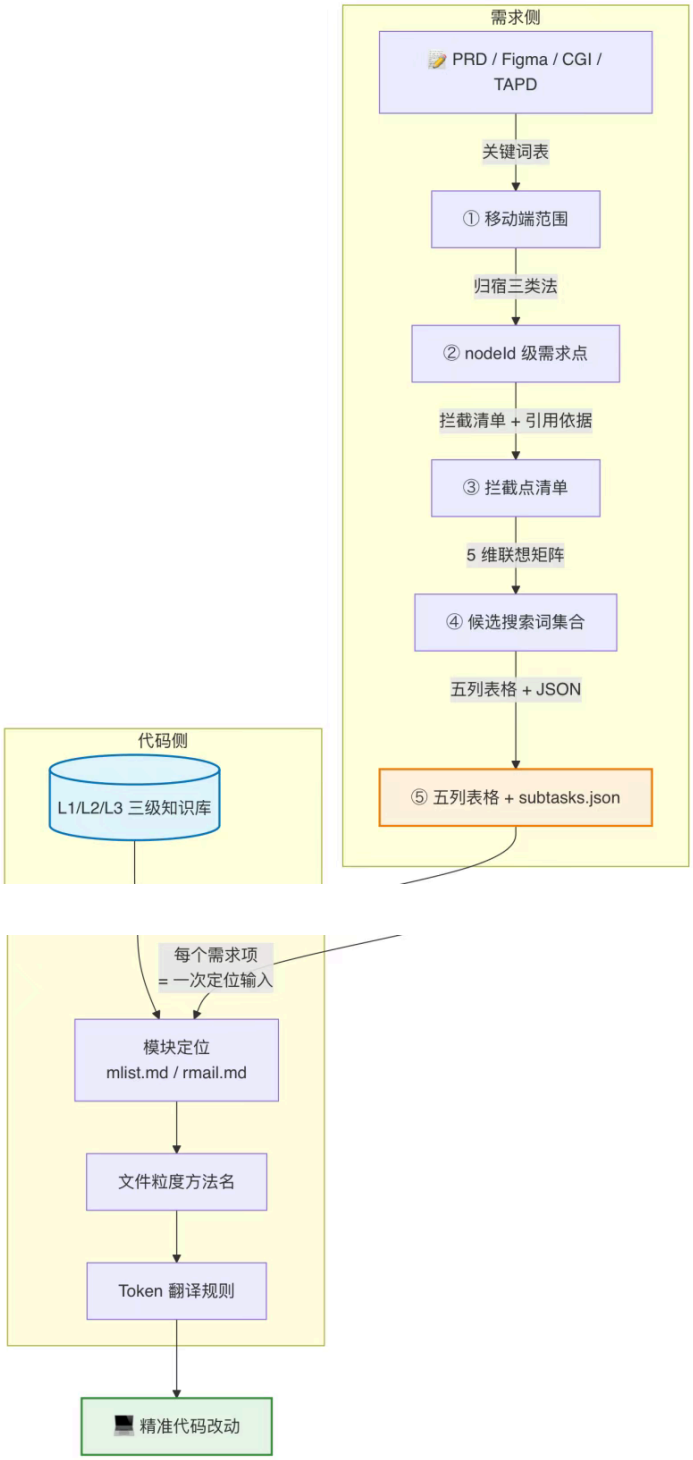


到这里，"产品语言 → 代码指令"的语义鸿沟就被彻底抹平了：

输入	经过 Skill 拆解后	给 AI 的指令变成
PRD 一段话："邮件列表顶部加红色提示条，点击跳转管理页"	M1 + M3 两个需求项	"在 `XYZTipsView.h/.mm`（来自 mlist.md L2 wiki）新增类型 `XYZMListTipsType_xxx`（参考已有枚举），点击响应跳转 `XYZWeeklyReportViewController`（来自 manager.md L2 wiki）"

6.7 完整翻译链：知识库 + 拆解规则 = 闭环

把第五章的代码侧地图、和本章的需求侧翻译合在一起看，就能看清 Skill 是怎么把"AI 独立开发需求"这件事工程化的：



两条链一对接，AI 就拥有了"独立开发完整需求"所需的全部确定性输入：

- **需求侧**：每个需求点是什么、范围在哪、关联设计稿哪个 nodeId、依据是什么
- **代码侧**：项目里有哪些模块、每个模块有哪些文件、每个文件做什么、UI Token 怎么翻译

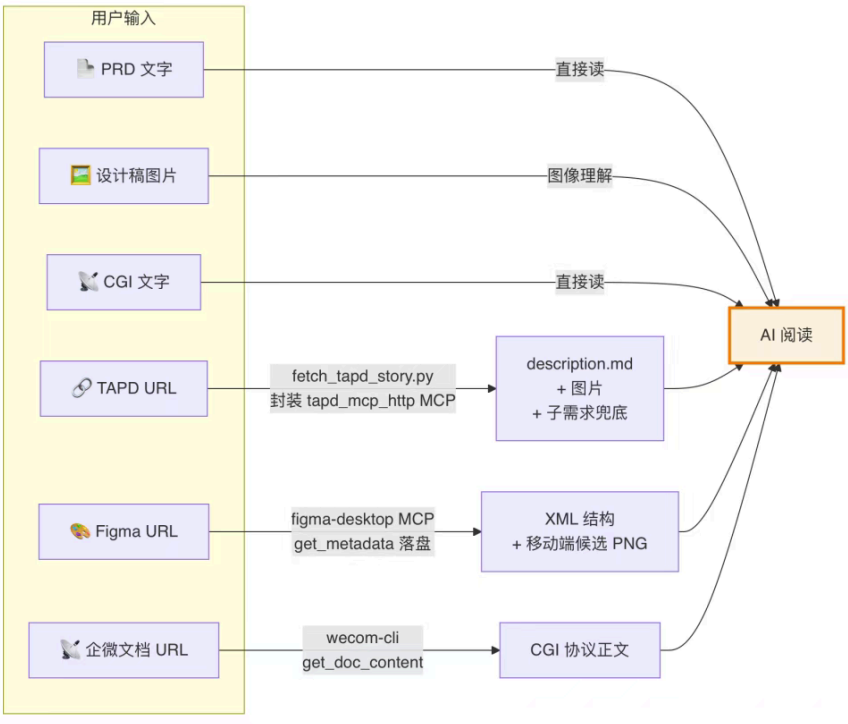
> 💡 **真正的提效不在"AI 写代码"，而在"AI 不再需要人来当翻译"**。当语义翻译这件事被规则化、可执行化、有产物可校验后，开发者从"PRD 翻译机"的角色里解放出来，转而成为"AI 的产品经理"——只在硬关卡处做决策。这就是 94% 代码生成率背后的真正机制。

七、公理 II：把"判断"留给 LLM，把"数据"交给脚本

LLM 最不擅长两件事：**精确数值**和**幂等执行**。Skill 把这两类工作全部下沉到脚本，LLM 只负责"读结果 + 下决策"。

7.1 多源物料收集：每种来源一个专用脚本

整个 Skill 支持六类输入，每类都有自己的"专用通道"，**严禁通用 `web_fetch`**：



为什么不能用 `web_fetch`？ 这正是 Skill 写死的 Critical 红线：

> ⛔ **RL-02** `doc.weixin.qq.com` 必须走 `wecom-cli`，`web_fetch` 鉴权后只拿到 HTML 外壳⛔ **RL-03** TAPD URL 必须走 `tapd_mcp_http` MCP，`web_fetch` 拿不到 markdown 描述

7.2 设计稿筛选：脚本直方图 vs LLM "手感"

Figma 一个 fileKey 下面常有几十上百个画板：海报、PC 端、平板、移动端、变体、注释稿……让 LLM 凭"看起来像移动端"挑出移动端是**灾难**。

Skill 的做法是：



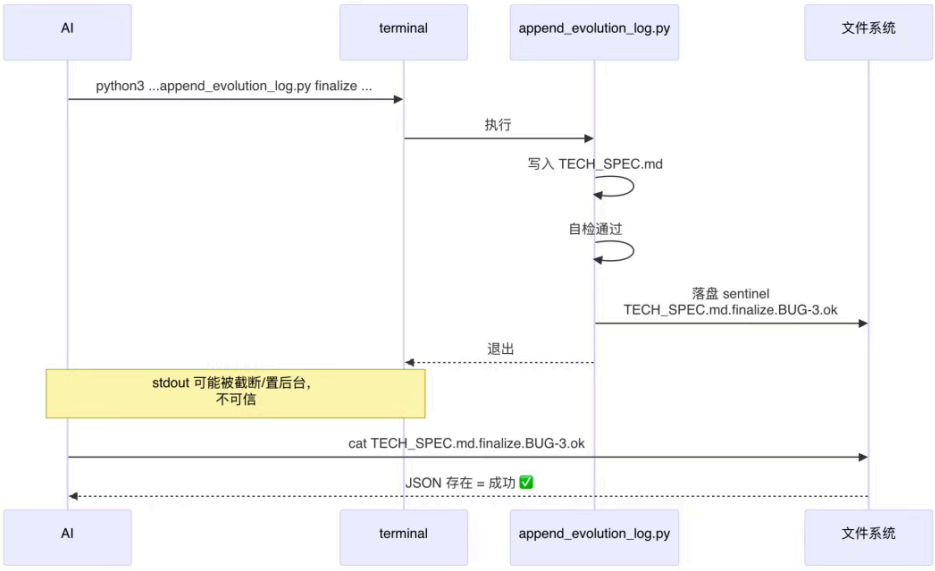
⛔ **RL-17：严禁 LLM 手工分桶——必须先跑 `scan_figma_frames.py` 出直方图（数据来自 `tools/iphone_sizes.json` 这份 iPhone 尺寸白名单），LLM 只能在已分桶基础上补判 `UNCERTAIN` 项，不能凭印象决定。**

这条红线把"AI 看图选稿"的随机性从根上扫掉了。

7.3 "落盘判定成功" — RL-32 的工程美感

`git commit` 长消息会被 terminal 当后台任务、stdout 会被截断、管道命令会变成异步……这些都是脚本和 LLM 之间常见的"信号丢失"陷阱。

Skill 引入了一个朴素但极漂亮的设计：**sentinel 文件 = 成功的唯一判据**。

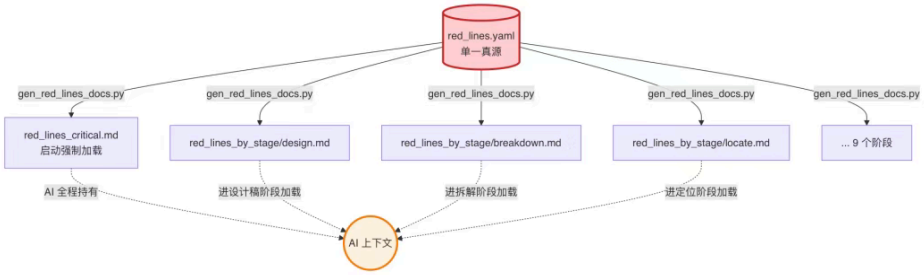


同样的思路也用在 `git commit`（RL-31：以 `git log -1 hash` 更新为唯一判据）。**任何"长跑命令"都不靠 stdout 报告成功，全靠落盘文件**——这是从无数翻车里淬出来的工程经验。

八、公理 III：红线机制——把"翻车"前置成"硬关卡"

LLM 在工程上最大的风险，是它"什么都敢说，什么都敢做"。Skill 用一套**红线系统**给它戴上紧箍。

8.1 红线架构：YAML 单一真源 + 分层加载



红线分两级：

- 🔴 **Critical**（6 条）：全流程必守，启动即加载，违反会**直接造成线上事故或严重返工**
- 🟡 **Standard**（30+ 条）：按阶段加载，违反会污染工程规范

8.2 触发即停 + 模板化报告

任何红线被触发，AI 必须停下并按固定模板汇报：

```
⛔ 触发红线 RL-XX：<标题>
当前情形：<具体说明>
建议处理：<回退到哪个步骤 / 需要用户确认什么>
```

这把"AI 偷偷做了它不该做的事"变成"AI 主动告诉你它撞上红线了"——**可观测性远比聪明更重要**。

8.3 几条"血泪换来"的 Critical 红线

红线	来源	设计思想
RL-15 编译必须通过	"AI 说做完了但其实编译都没过"	退出码 0 = 唯一判据；自修复硬上限 3 轮
RL-16 未按阶段执行	"用户一句话指令 → AI 跳过拆解直接改代码 → 越界"	后一阶段输入 = 前一阶段产出
RL-13/14 先看后写、模仿已有	"AI 发明新模式 → 项目里独此一家"	先通读完整方法 + 搜索同类分支
RL-31 commit 同步执行	"长 commit 消息被 terminal 置后台 → AI 误判失败重提"	`git log -1` hash 更新是唯一成功证据

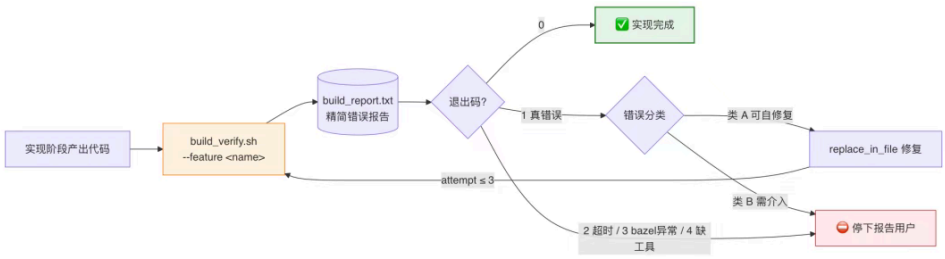
> 红线只是把"翻车点"拉到了硬关卡，但还有一个更根本的问题：**AI 怎么证明自己写的代码真的"做对了"？** 编译过 ≠ 跑得对，跑得起 ≠ 长得对。下一节我们讲 Skill 怎么把"代码质量验证"也工程化、自动化。

九、运行时验证：让 AI 自己跑通

> AI 最大的诚信问题是"自报完成"——说"已经做完了"，结果编译都没过；说"功能正常"，截图打开一看 UI 错位。Skill 把"验证"拆成两道闸门：**编译验证**（代码层）+ **模拟器验证**（运行时 + 视觉），两道都通过才允许进入沉淀阶段。

9.1 闸门一：编译验证——退出码 0 是唯一判据

代码改完后，AI 不允许说"实现完成"——必须先跑通 `bazel build` 。



A/B 分类的设计精髓：

类别	典型场景	AI 能否自处理
A 可自修复	缺分号 / 标识符未声明 / 类型不匹配 / 枚举漏 case / `#import` 找不到	✅ 直接 `replace_in_file` 修，重跑编译
B 需用户介入	`BUILD.bazel` 配置错 / 链接错误 / 错误位于三方 framework / 错误文件不在本次改动集合	⛔ 立即停下，绝不硬试

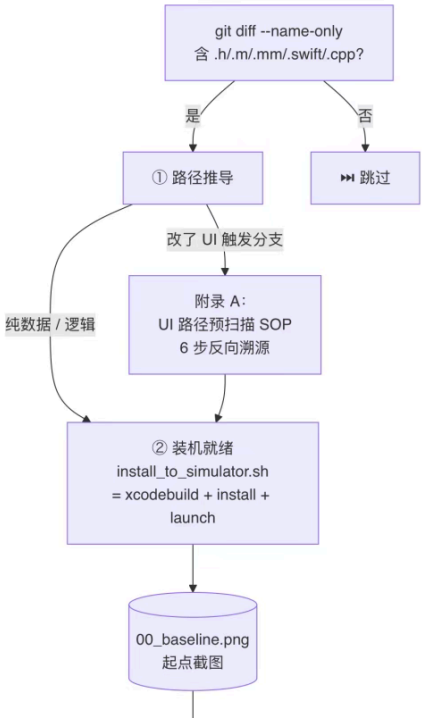
> ⛔ **RL-15 + 自修复硬上限 3 轮**：超过 3 轮仍编译不过 → 强制停下报告用户，不允许继续。这条规则把"AI 越改越乱"的死循环锁死。

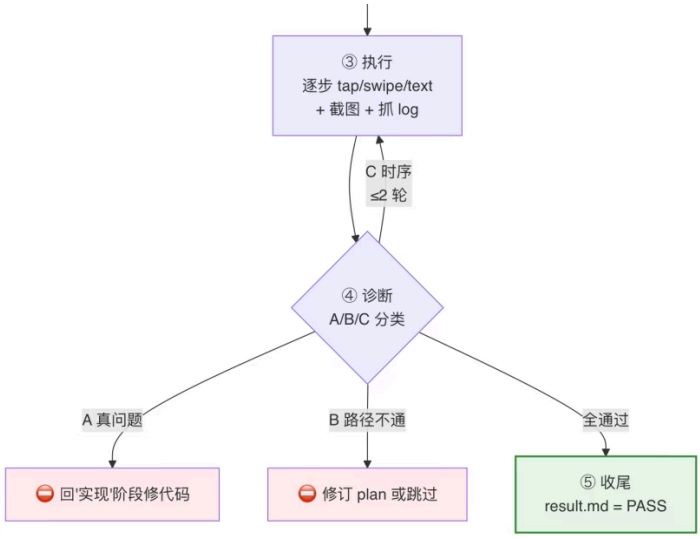
报告里直接带上下文代码行——让 AI 不用回头读源码就能修。这是脚本设计的一个小巧思：

```
App/Mailbox/mailcore/mailbox_protocol.cpp:1822:25:
  error: use of undeclared identifier 'undefined_xxx'
 1822 | void __test_error__() { undefined_xxx(); }
      |                         ^^^^^^^^^^^^
```

9.2 闸门二：模拟器验证——真跑一遍 + 截图核对

编译通过 ≠ 功能正确。Skill 用一套**自动化 UI 验证流程**让 AI 自己装机、自己点击、自己截图、自己核对预期。





第①步：路径推导——从 git diff 反推一条 UI 路径

AI 不是"想点哪点哪"，而是按 `git diff` 改动 + TECH_SPEC §3「相关代码位置」+ 设计稿终态图，**反推出一条具体的 UI 验证路径**：

改动类型	验证终点
改 UI（View / Controller）	该 UI 的真实可见状态（截图能看到）
改数据 / 解析	UI 上能体现该数据的页面 + 抓日志确认数据流
改纯逻辑（无 UI 直接体现）	XYZLOG_WARN 日志关键字命中

verify_plan.md 的标准骨架：

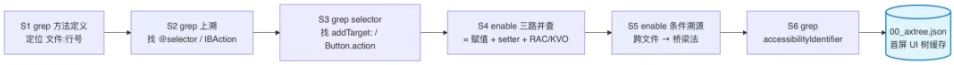
```
# 模拟器验证计划：<feature-name>

## 操作步骤
1. launch App                                → 01_launched.png
2. tap 邮件 Tab                               → 02_mail_tab.png
3. tap 第一封邮件                              → 03_detail.png
4. 观察顶部 Tips 文字是否含 "xxx"               → 04_tips.png
5. tap nav_back_arrow                        → 05_back.png

## 预期
- 步骤 4 截图中 Tips 文字 == "<期望文案>"
- runtime.log 中 `XYZLOG_WARN(@"mailbox xxx")` 命中 ≥ 1
```

UI 路径预扫描：6 步反向溯源（附录 A 的精华）

如果改动涉及"按钮 enable 条件 / 拦截弹窗 / 新增点击响应"，AI 必须先做一次**预扫描**，把"代码层方法名"反推到"UI 层可点击控件"，避免点错或点了没反应：



> 📌 **桥梁法——当依赖变量跨文件赋值时，按 3 类桥梁定位源头：通知（`postNotificationName:`）/ KVO（`RACObserve(`）/ Delegate（`<DelegateProtocol> =`）。这是把"AI 找不到控件来源"这种顽疾规则化的关键。**

第③④步：执行 + A/B/C 三类诊断

每步固定 5 个动作，**实时汇报，不静默连跑**：

```
📍 步骤 N/M: <动作>
- 命令: idb ui tap --udid $UDID 200 420
- 截图: 03_detail.png
- 观察: 导航栏标题 "邮件详情", Tips 区域可见
```

预期点核对失败时，按 **A/B/C 分类**分流：

类	现象	处理
A 真问题（代码 bug）	期望 UI 没出现 / 字段值错 / `assert|crash|Error` 命中	⛔ 本阶段不修代码，回「实现」阶段
B 路径不通（验证设计错）	被登录页 / 引导页拦截 / 当前帐号没数据 / 真机才能复现	⛔ 修订 verify_plan 或跳过
C 脚本/时序（可自修复）	元素未渲染就 tap / 坐标算错 / 输入法没切到位	✅ 阶段内重试 ≤ 2 轮

> 🎯 **设计精髓**：A/B/C 分类把"该不该重试"这个糊涂账变成清晰决策。AI 不允许在 A/B 类问题上反复硬试，**最多 2 轮 C 类重试不过 → 升级为实质性问题报告用户**。

第⑤步：视觉对齐核对——RL-30 的硬关卡

> ⛔ **RL-30**：触发了 RL-29（UI 改动）但 `ui_alignment_spec.md` 不存在 / 未对齐项 ≥1 → 视觉对齐**直接判 FAIL**，不允许跳过。

光"截图能看到"还不够，UI 改动还要**逐项核对数值**：

```
## 视觉对齐核对（依据 ui_alignment_spec.md，RL-30）

- [x] XYZTopicEmptyFooter container.height = 280 ✅ （截图实测 280）
- [x] icon 居中且 size 96×96 ✅
- [x] title 字号 16 / Medium ✅
- [△] desc lineHeight 偏小 1pt（已知偏差，spec 已记录）
```

```
- [x] cta 主蓝色 ✅

## 视觉对齐结论
- 关键差异 0 / 接受偏差 1 / **未对齐 0**
- 未对齐 ≥1 → 状态自动降级为 ❌ FAIL
```

这把"设计稿走样"这个 UI 工程顽疾彻底显式化——**不再依赖测试同学手肉眼比对，而是 AI 自己拿着数值清单逐项核对。**

9.3 那些"血泪换来"的运行时小坑

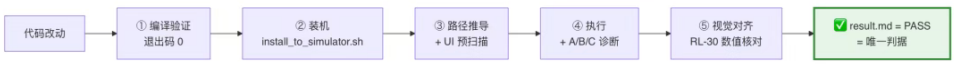
模拟器验证过程踩过不少坑，Skill 把它们沉淀成 `simulator_toolbox.md` 里的**死角清单**——这些是 AI 必须知道的"不能做什么"：

死角	为什么不行	替代方案
边缘左滑返回	`UIScreenEdgePanGestureRecognizer` 要求真实 `touchDown→hold→move` 时序，`idb ui swipe` 是合成事件，**模拟器永远识别不出**	找 `nav_back_arrow` 的 AX 标识 + tap
3D Touch / 力度长按	模拟器不支持力度感应	用菜单按钮 / 开 debug 后门
物理像素 ↔ 逻辑像素	截图是物理像素，`idb ui tap` 吃逻辑像素，硬编码坐标必错	`scale = logical_w / physical_w` 动态换算
登录态丢失	`simctl uninstall` 清沙盒会丢登录	同 bundle id `simctl install` 不动沙盒，登录态保留
shell heredoc 里 Python f-string `!r`	zsh 把 `!r` 当 history expansion → 命令变乱	改用 `repr(x)` 或独立 `.py` 文件

> 这些坑没有一条是"模型不够聪明"导致的——全是工程层面的真实陷阱。沉淀成手册之后，每个新会话的 AI 都能直接绕开。

9.4 验证闭环：从代码改动到"敢说做完了"

把两道闸门串起来看，AI 从"改完代码"到"敢说做完了"经历了 5 道把关：



每一道关都有**机器可校验的产物**：`build_report.txt` 退出码、`<NN>_xxx.png` 截图、`runtime.log` 日志命中、`result.md` 状态字段。**全部由文件证明，不靠 AI 自报。**

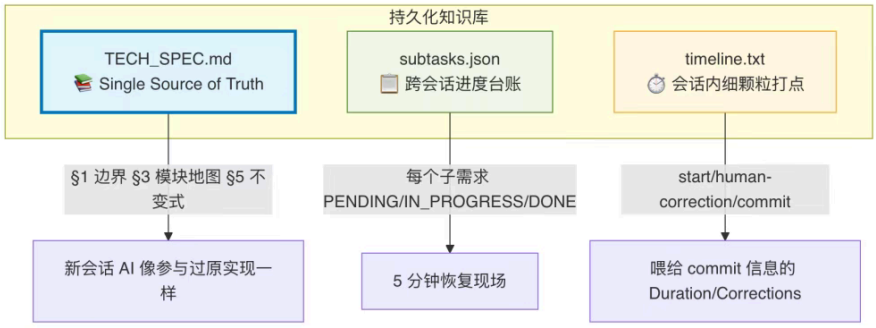
> 💡 **本质思想**：把"质量保障"这件事从"靠测试同学发现 bug"变成"AI 自己写代码自己验证自己交差"。这才是 AI 能从"辅助"升级为"主导"的关键——**当 AI 拿出来的不仅是代码，**

> **还有截图、日志、视觉对齐报告时，开发者只需要做最后一道 review，而不是手动跑一遍验证。**

十、公理 IV：跨会话知识传承——TECH_SPEC.md 是灵魂

如果说前三公理解决的是"一次会话内的提效"，那这条公理解决的是**真正让 AI 像团队成员一样工作**——会做、能记、可接力。

10.1 三件套：分别承担不同尺度的"记忆"



文件	时间尺度	内容
`TECH_SPEC.md`	永久	功能边界、模块地图、不变式、Bug/迭代演进历史
`subtasks.json`	跨会话	每个子需求的状态、当前阶段、关联 commit
`timeline.txt`	会话内	`start` / `human-correction` / `commit` 三类事件流水

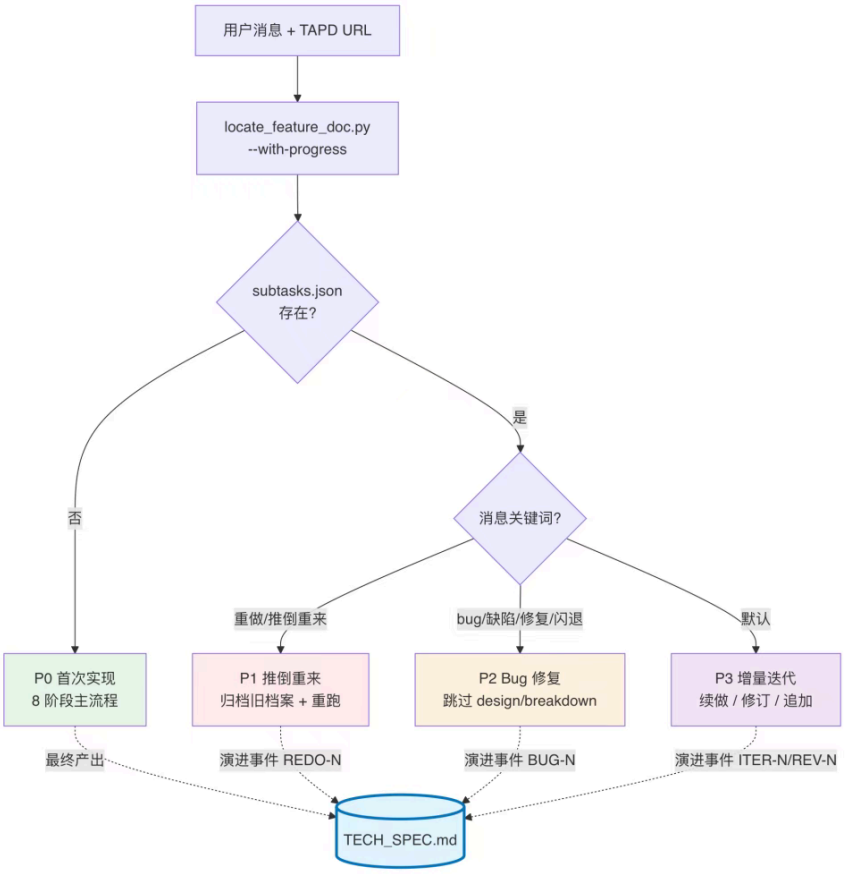
10.2 TECH_SPEC.md 的章节结构（精华）

```
§0  AI 自检清单          ← 给下次会话的 AI 当"入场扫描"
§1  功能边界             ← 哪些做、哪些不做（防越界）
§3  模块地图             ← 文件 + 关键方法 + 调用链
§5  不变式               ← 不能动的命名、文件清单、拦截边界
§7  演进事件             ← 按时间线排列的 BUG-N / ITER-N / REV-N
§8  产物清单             ← 每次 commit 改了什么
§9  版本号               ← v1.0 → v1.1 → ... → v2.0 (baseline 合并)
```

新会话的 AI 只要按 §0 → §1 → §3 → §5 → §7 顺序读完，就能"无缝接力"。

10.3 四类入口：根据现场状况自动分流

这套接力机制配合 4 种入口，把"需求开发"覆盖到了完整生命周期：



> 同一个 TAPD 需求的整个生命周期——从首次实现到 N 轮迭代、M 个 bug 修复、偶尔的推倒重来——全部由这一份 `TECH_SPEC.md` 串联起来。

10.4 硬关卡 HK：信任但不放任

每个工作流里都嵌着若干**人机硬关卡**（Hard Checkpoint），强制要求用户确认：

硬关卡	触发时机	用户回什么
HK-0 现场快报	接力入口进入后第一时间	确认进度 / 改 N
HK-1 PENDING 条目	§7 翻译完成后	"确认 / 改 xxx"
HK-2 沉淀 ok	TECH_SPEC 落盘前	"沉淀 ok / 通过"
HK-3 commit 文案	git commit 前	"提交 / go"

这套"硬关卡"是 Skill 工程的精髓之一——**自动化和可控性的平衡点**：AI 跑得飞快，但任何一个不可逆动作都先让人点头。

十一、提效效果：到底快了多少？

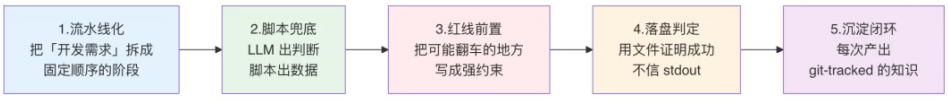
> 数据来源于本项目近半年实际跑下来的体感（非严格 benchmark），仅供参考。

环节	传统方式	Skill 方式	提效来源
需求拆解	1~2 小时（看 TAPD / Figma + 整理）	5~10 分钟	多源脚本一站式拉取 + 自动归宿校验
代码定位	30 分钟~2 小时（grep 试错）	5~15 分钟	五步定位法 + project_wiki 索引
实现	视复杂度	-30%~50%	"先看后写 + 模仿已有"减少返工
编译自查	手动来回	自动 3 轮	`build_verify.sh` + 自修复
UI 验证	手动装机点击 + 肉眼比对	自动装机 + 截图 + 视觉对齐	`install_to_simulator.sh` + A/B/C 诊断 + RL-30 数值核对
Bug 修复接力	重新读代码 1+ 小时	5 分钟恢复现场	TECH_SPEC.md + subtasks.json
提交规范	手写 commit 三段	自动渲染 + 人工确认	时间线 + 模板

最大的隐性收益：**新人 / AI 都能直接接手已有需求的迭代**，不再依赖"问原作者"。这是 `TECH_SPEC.md` 带来的复利效应。

十二、关键启示：如果你也想做这种 Skill

我们踩过的坑收敛成 5 条原则，普适性强，建议复用到你自己的项目：



原则	一句话总结
流水线化	把"需求开发"拆成 8 个语义化阶段，每个阶段输入/输出/退出标准都可机器校验
脚本兜底	LLM 负责"读判断"，精确数值/幂等执行/批量操作全部下沉到 Python/Shell 脚本
红线前置	把"哪些事 AI 绝对不能做"写成 YAML + 分层加载，触发即停、模板化报告
落盘判定	任何长跑命令的成功证据都是"文件存在"，不依赖 stdout（terminal 会截断/置后台）
沉淀闭环	每次需求产出一份 git-tracked 的 `TECH_SPEC.md`，让"知识"和"代码"等量齐观

十三、附录：Skill 目录速览

整套 Skill 由 6 大组件构成，按"AI 进入流水线"的视角分层组织：

```
skills/mailplugin-feature-dev/
|
├── ① 对外入口（LLM 启动时加载）
```

```
|   ├── SKILL.md                          # 流程总图 + 4 类入口分流 + 强约束
|   ├── README.md                         # 给人看的使用指南
|   └── CHANGELOG.md                      # 版本变更日志
|
├── ② 安装与配置
|   └── setup/
|       ├── install.sh                    # 一键安装（含 MCP 注册、依赖检测）
|       ├── uninstall.sh                  # 一键卸载
|       └── mcp.tapd.json                 # TAPD MCP Server 配置
|
├── ③ 自动化脚本（"判断交给 LLM，数据交给脚本"）
|   └── tools/
|       |  -- 收料（公理 II：绕过上下文截断）--
|       ├── fetch_tapd_story.py           # TAPD 一站式收料：单据+附件+评论
|       ├── fetch_tapd_images.py          # TAPD 图片批量下载
|       ├── fetch_figma_mcp.py            # Figma MCP 数据落盘
|       ├── scan_figma_frames.py          # 设计稿直方图筛选（RL-17）
|       |
|       |  -- 文档生成与维护 --
|       ├── locate_feature_doc.py         # 定位 TECH_SPEC.md 路径
|       ├── render_tech_spec.py           # TECH_SPEC.md 首次渲染
|       ├── append_evolution_log.py       # §7/§8/§9 增量维护 + sentinel
|       ├── append_bug_fix.py             # bug 修复记录追加
|       ├── breakdown_subtasks.py         # 子任务台账（跨会话接力）
|       ├── gen_red_lines_docs.py         # 红线 yaml → 派生 md
|       |
|       |  -- 编译与验证（公理 I：落盘判定）--
|       ├── build_verify.sh               # bazel 编译 + 报告
|       ├── check_implement_done.sh       # 实现完成度自检
|       ├── check_intermediate_artifacts.py # 阶段产物完整性检查
|       ├── check_project_wiki_stale.py   # 知识库时效性扫描
|       ├── check_ui_token_usage.sh       # UI Token 合规检查
|       |
|       |  -- 模拟器与提交 --
|       ├── install_to_simulator.sh       # 安装包到模拟器
|       ├── iphone_sizes.json             # 设备尺寸数据库
|       ├── finalize_commit.sh            # 提交收尾
|       ├── render_commit_msg.py          # commit message 模板渲染
|       ├── timeline_to_commit_lines.py   # 时间线 → commit 行
|       └── md_to_pdf.py                  # 文档导出
|
├── ④ 知识库与映射（"代码侧地图 + 语义桥"）
|   └── references/
|       ├── project_wiki/                 # 分模块知识库（按业务域 + 基础设施分册）
|       |   ├── overview.md               # 总览索引（< 5KB，作为 L1 入口）
|       |   └── *.md                      # 各模块 L2 详情（按需加载）
|       |
```

```
|       ├── figma_token_mapping.md        # L3 语义桥：Figma → 工程代码
|       ├── figma_device_sizes.md         # 设计稿设备尺寸映射
|       └── ui_components_wiki.md         # 统一 UI 组件文档
|
├── ⑤ 流程细则（按需加载，不污染上下文）
|   └── references/
|       |  -- 8 个阶段完整执行细则 --
|       ├── stage_locate.md               # 阶段 1：意图消歧 + 定位
|       ├── stage_design.md               # 阶段 2：设计文档收料
|       ├── stage_breakdown.md            # 阶段 3：需求拆解 + 子任务台账
|       ├── stage_implement.md            # 阶段 4：编码实现
|       ├── stage_verify.md               # 阶段 5：编译验证
|       ├── stage_simulator_verify.md     # 阶段 6：模拟器验证
|       ├── stage_commit.md               # 阶段 7：提交收尾
|       ├── stage_archive.md              # 阶段 8：归档与沉淀
|       |
|       |  -- 4 类入口子流程 --
|       ├── bug_fix_workflow.md           # 入口 ②：bug 修复
|       ├── incremental_workflow.md       # 入口 ③：增量迭代
|       ├── redo_workflow.md              # 入口 ④：推倒重来
|       |
|       |  -- 工具箱 --
|       ├── simulator_toolbox.md          # 模拟器调试工具箱
|       └── tech_spec_template.md         # TECH_SPEC.md 模板
|
└── ⑥ 红线机制（公理 III：硬关卡）
    └── references/
        ├── red_lines.yaml                # 红线单一真源（DSL）
        ├── red_lines_critical.md         # 全局强制加载（启动即生效）
        └── red_lines_by_stage/           # 分阶段按需加载
            ├── global.md                 # 跨阶段通用红线
            ├── locate.md                 # 阶段 1 红线
            ├── design.md                 # 阶段 2 红线
            ├── breakdown.md              # 阶段 3 红线
            ├── implement.md              # 阶段 4 红线（最厚一份）
            ├── verify.md                 # 阶段 5 红线
            ├── simulator_verify.md       # 阶段 6 红线
            ├── commit.md                 # 阶段 7 红线
            └── archive.md                # 阶段 8 红线
```

> 一个直观感受：**`references/` 比 `tools/` 体量更大**——这是"AI 提效在工程而不在模型"最朴素的证据，绝大部分能力都来自被显式编写的规则、知识、模板，而不是"指望模型聪明"。

写在最后

我们一开始想做的是"让 AI 帮我写代码"；做完才意识到——**真正有价值的，是让"需求开发"这件事本身被显式建模、可观测、可接力。**

Skill 只是把这些工程规范"具象成了 LLM 能消化的格式"。而沉淀下来的 `TECH_SPEC.md` 和 `project_wiki`，即使有一天换掉 AI，对人也是同样有用的资产。

AI 提效的天花板，既在模型，也在工程。



腾讯技术工程
腾讯技术官方号。腾讯技术创新、前沿领域发布解读平台。
688篇原创内容
公众号



阅读 5.0万

留言 1

写留言



asc 广东 7月23日

31

一个xxx跑通xxx的，一律当忽悠